

Virtual Machines, Virtual Machine Monitors and Virtualization Technology

Difference between Traditional Computer and Virtual machines

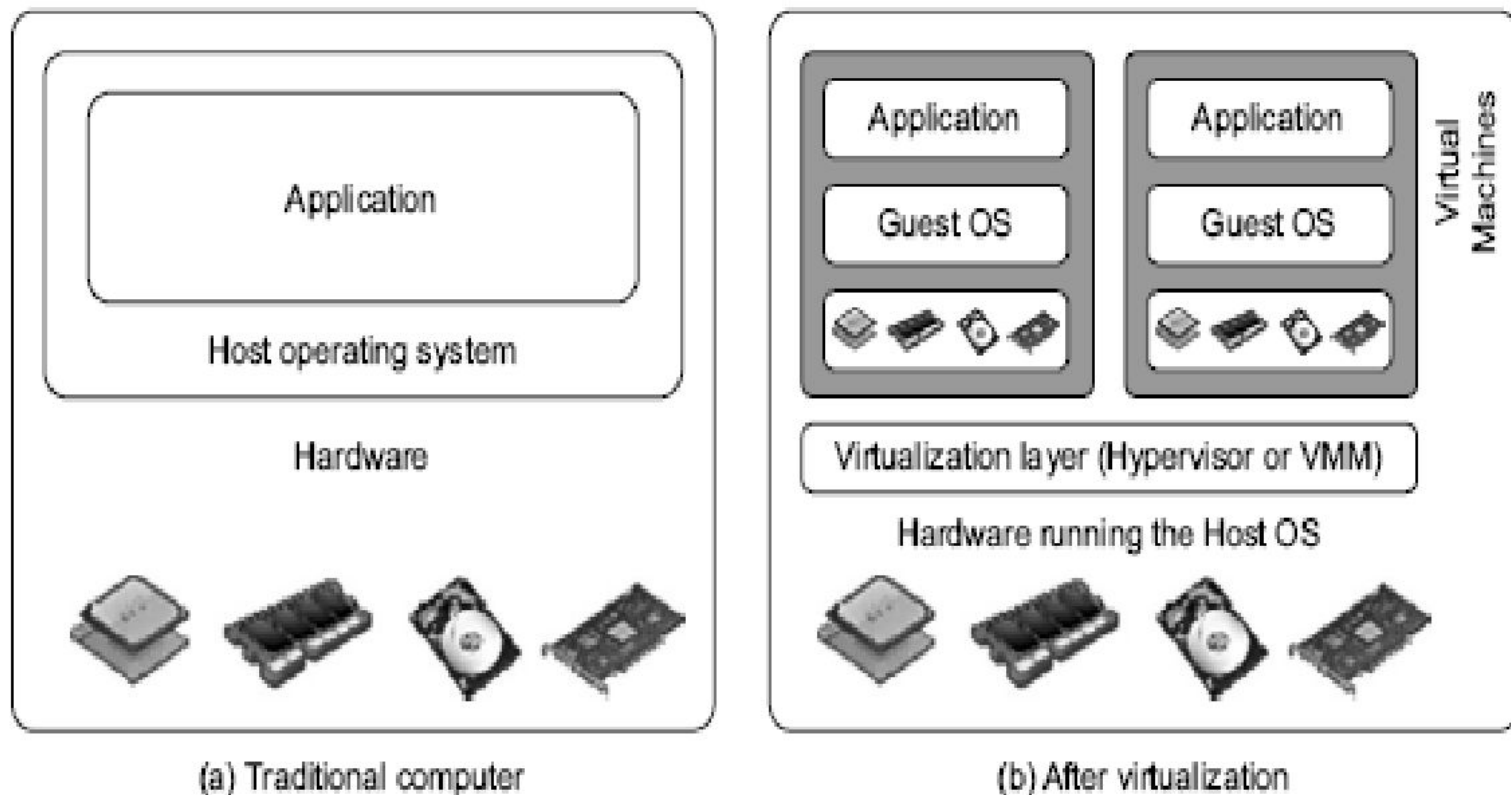
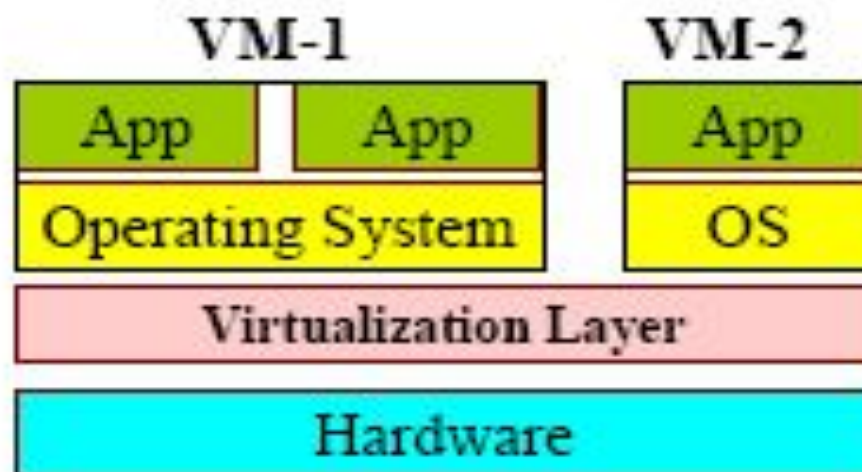


FIGURE 3.1

The architecture of a computer system before and after virtualization, where VMM stands for virtual machine monitor.

Virtualization is:

- A level of indirection between hardware and software.



- Virtual Machine abstraction
 - Run all software written for physical machine.

Virtual Machine, Guest Operating System, and VMM (Virtual Machine Monitor) :

Virtual Machine

A representation of a real machine using software that provides an operating environment which can run or host a guest operating system.

Guest Operating System

An operating system running in a virtual machine environment that would otherwise run directly on a separate physical system.

The Virtualization layer is the middleware between the underlying hardware and virtual machines represented in the system, also known as *virtual machine monitor* (VMM).

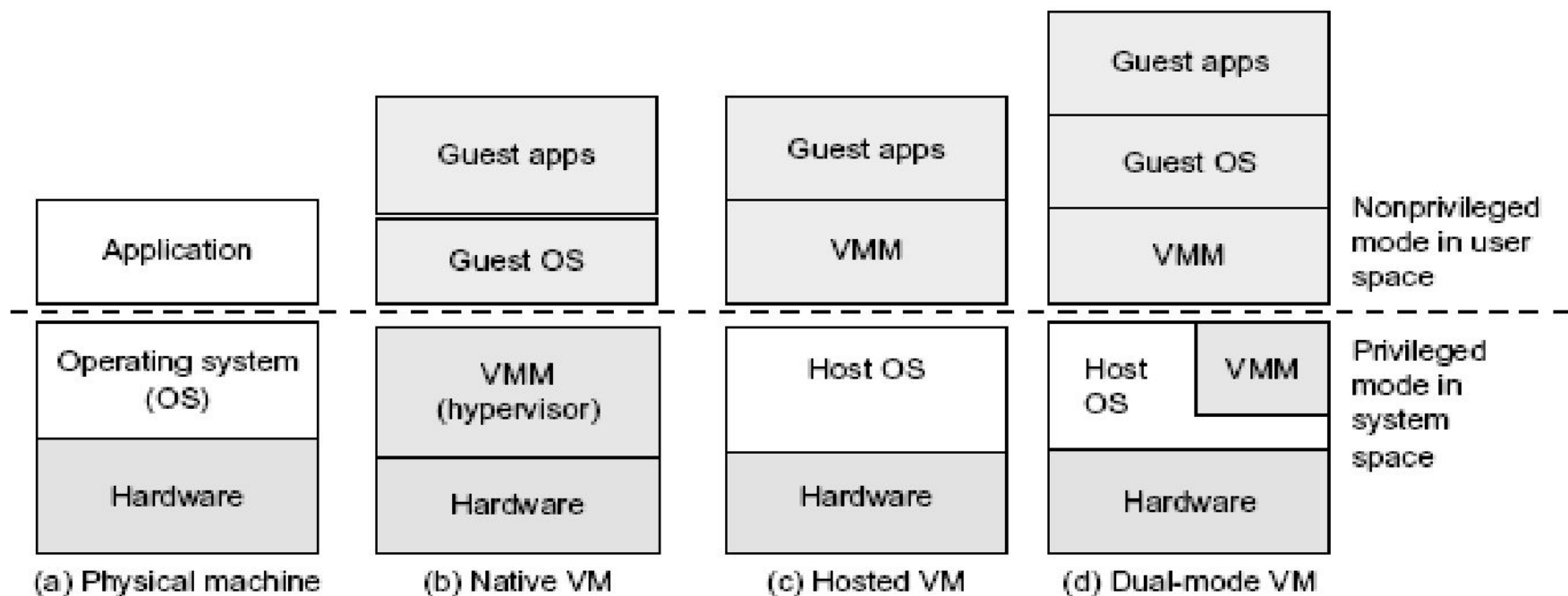


FIGURE 1.12

Three VM architectures in (b), (c), and (d), compared with the traditional physical machine shown in (a).

Type	Runs on Hardware?	Speed	Example
Physical	Yes	Fast	Normal PC
Native VM	Yes (direct)	Very fast	VMware ESXi
Hosted VM	No (via OS)	Slower	VirtualBox
Dual-mode	Partial	Medium	Advanced systems

Physical = No VM

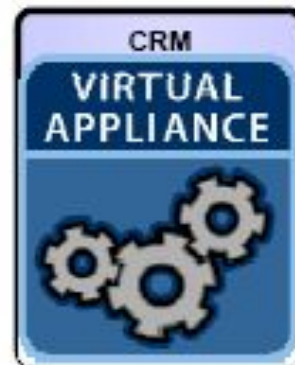
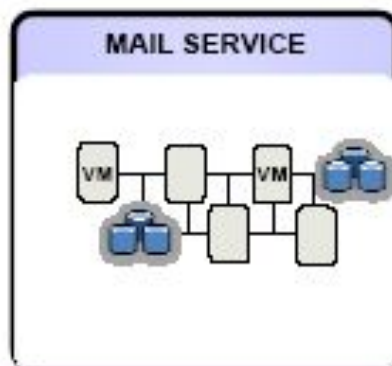
Native VM = VM directly on hardware

Hosted VM = VM on top of OS

Dual-mode VM = Mix of both

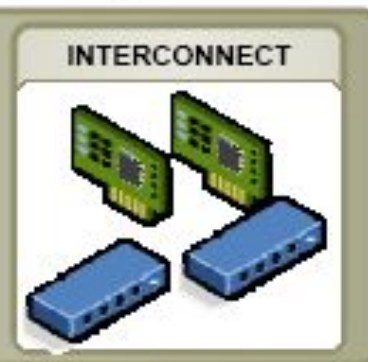
User's view of virtualization

LOGICAL VIEW



Virtualization Layer - Optimize HW utilization, power, etc.

PHYSICAL VIEW



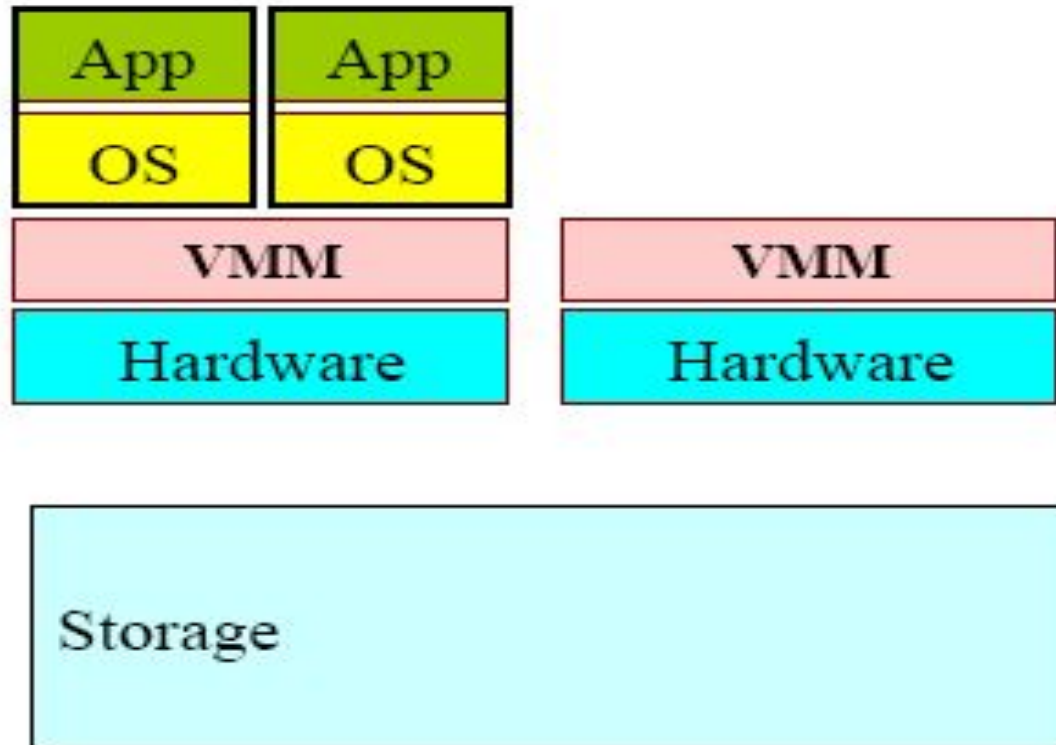
VMM : Virtual Machine Monitor

Virtual Machine Monitor

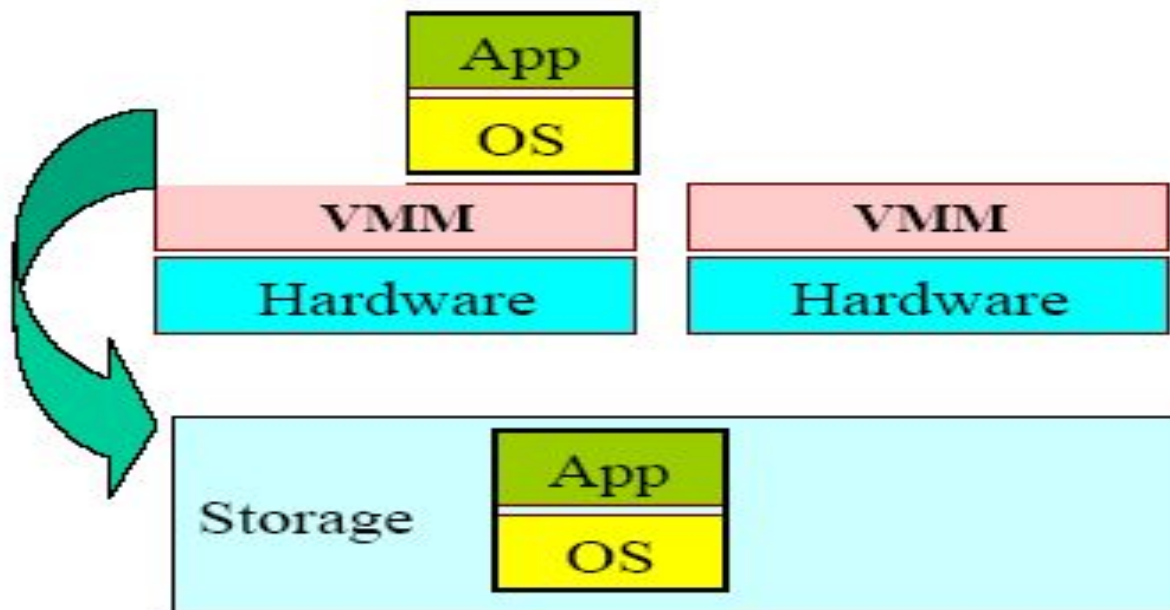
Software that runs in a layer between a hypervisor or host operating system and one or more virtual machines that provides the virtual machine abstraction to the guest operating systems. With full virtualization, the virtual machine monitor exports a virtual machine abstraction identical to a physical machine, so that standard operating systems (e.g., Windows 2000, Windows Server 2003, Linux, etc.) can run just as they would on physical hardware.

Low-Level VMM Operations (1)

- Multiplex

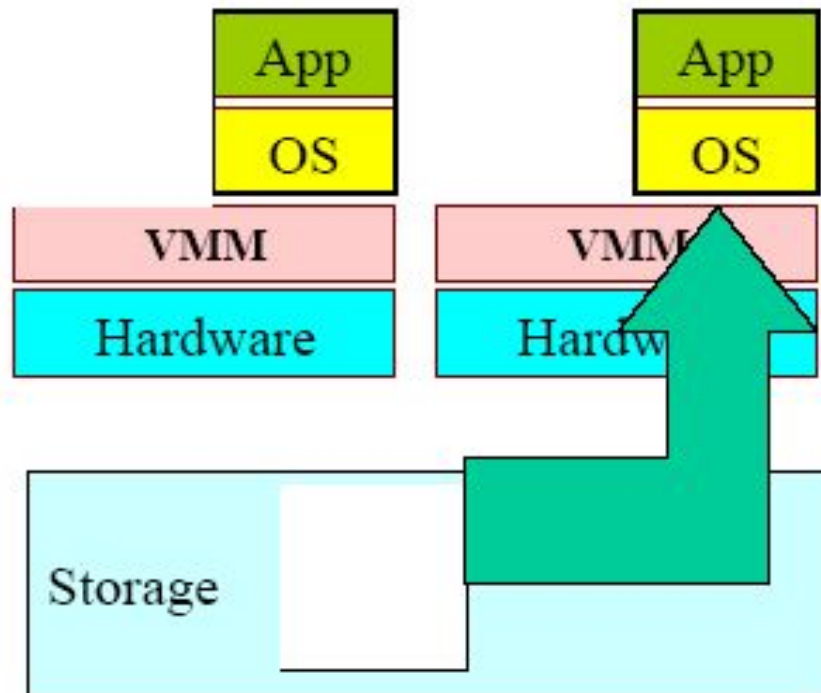


Low-Level VMM Operations (2)



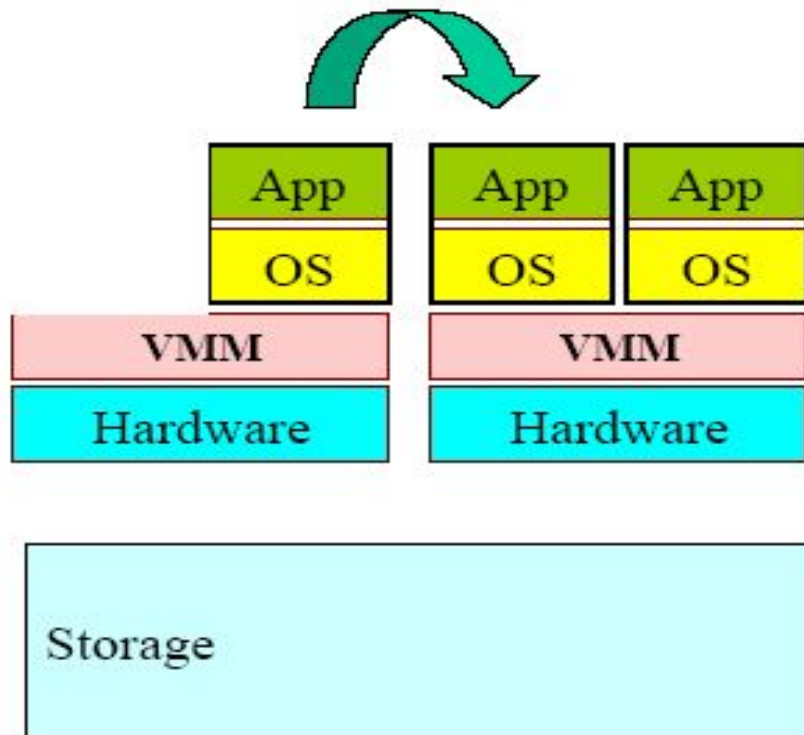
- Multiplex
- Suspend

Low-Level VMM Operations (3)



- Multiplex
- Suspend
- Resume (Provision)

Low-Level VMM Operations (4)



- Multiplex
- Suspend
- Resume (Provision)
- Migration

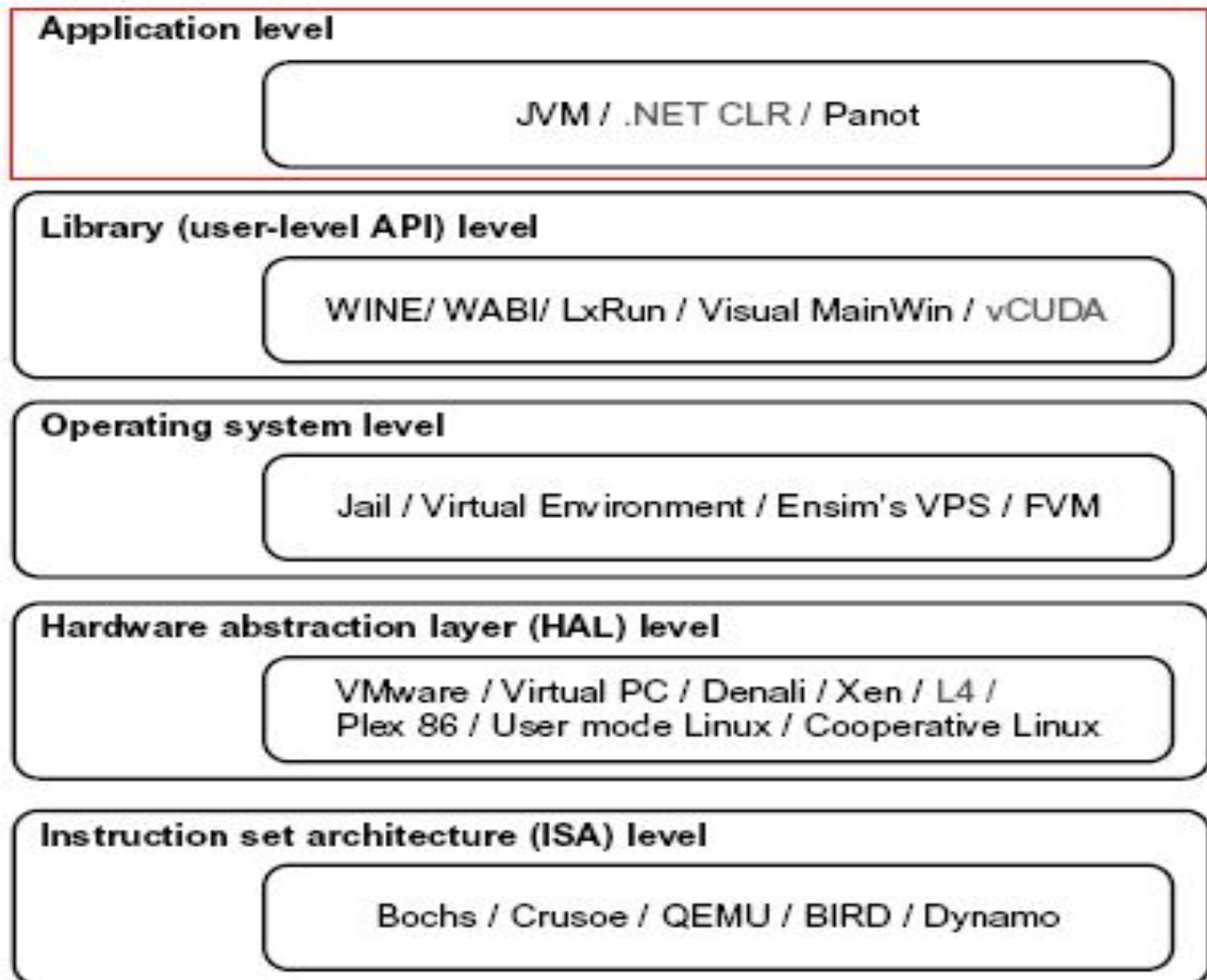


FIGURE 3.2

Virtualization ranging from hardware to applications in five abstraction levels.

Virtualization at ISA level:

Virtualization is performed by Emulating a given ISA by the ISA of the host machine. For example, MIPS binary code can run on an x-86-based host machine with the help of ISA emulation. Typical systems: Bochs, Crusoe, Qemu, BIRD, Dynamo

Advantage: It can run a large amount of legacy binary codes written for various processors on any given new hardware host machines; Instruction set emulation leads to virtual ISAs created on any hardware machine, i.e. best application flexibility.

Virtualization at ISA level:

Shortcoming & limitation: The basic emulation method is through **CODE INTERPRETATION**.

An interpreter program interprets the source instructions to target instructions one by one.

- One source instruction may require tens or hundreds of native target instructions to perform its function, which is relatively slow.
- For better performance, **DYNAMIC BINARY TRANSLATION** is desired (translates basic blocks of dynamic source instructions to target instructions).
- Instruction Set emulation requires binary translation and optimization.
- A Virtual-ISA requires adding a processor-specific software translation layer in the compiler.

Virtualization at Hardware Abstraction level:

- Virtualization is performed right on top of the bare hardware.
- It generates virtual hardware environments for VMs, and manages the underlying hardware through virtualization.
- Typical systems: VMware, Virtual PC, Denali, Xen

Advantage: has higher performance and good application isolation

Shortcoming & limitation: very expensive to implement (complexity)

Virtualization at Operating System level:

- ▣ It is an abstraction layer between traditional OS and user applications.
- ▣ This virtualization creates isolated containers on a single physical server and the OS-instance to utilize the hardware and software in datacenters.
- ▣ The containers behaves like real servers.
- ▣ OS –level virtualization is commonly used in creating virtual hosting environment to allocate hardware resources among a large number of mutually distrusting users.
- ▣ Also used in consolidating servers.
- ▣ Typical systems: Jail / Virtual Environment / FVM

Advantage: have minimal startup/shutdown cost, low resource requirement, and high scalability; synchronize VM and host state changes.

Shortcoming & limitation: all VMs at the operating system level must have the same kind of guest OS; poor application flexibility and isolation.

Virtualization at OS Level

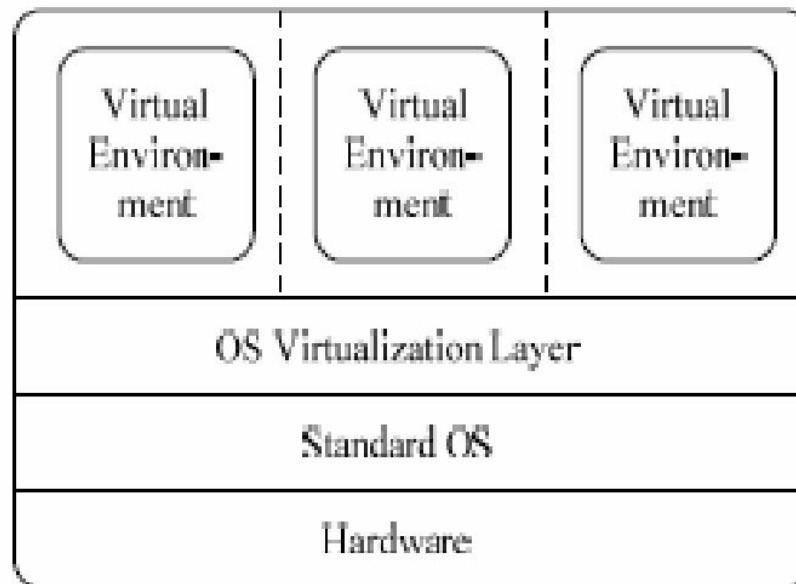


Figure 6.3 The virtualization layer is inserted inside an OS to partition the hardware resources for multiple VMs to run their applications in virtual environments

Virtualization for Linux and Windows NT Platforms

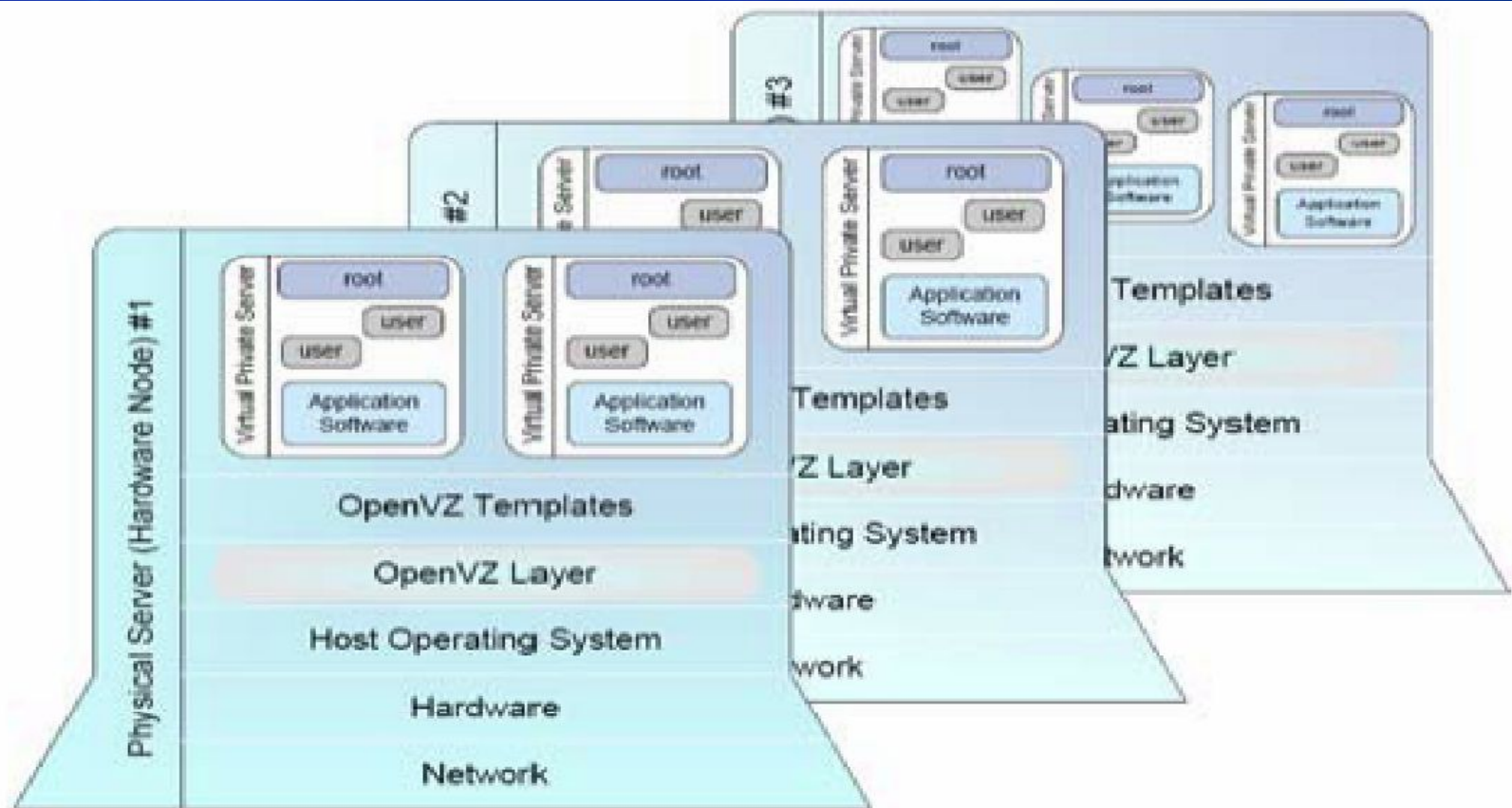


Figure 6.4 OpenVZ inserts a virtualization layer called OpenVZ inside the host OS. This layer provides some OS images to create VMs quickly (Courtesy of OpenVZ User's Guide, <http://ftp.openvz.org/doc/OpenVZ-Users-Guide.pdf>)

Advantages of OS Extension for Virtualization

1. VMs at OS level has minimum startup/
shutdown costs
2. OS-level VM can easily synchronize with its
environment

Disadvantage of OS Extension for Virtualization

All VMs in the same OS container must have the same or similar guest OS, which restrict application flexibility of different VMs on the same physical machine.

Library Support level:

Most applications use APIs exported by user-level libraries rather than using lengthy system calls by the OS.

- Since most systems provide well-documented APIs, such an interface becomes another candidate for virtualization.
- It creates execution environments for running alien programs on a platform rather than creating VM to run the entire operating system.
- It is done by API call interception and remapping. Typical systems: **Wine, WAB, LxRun , VisualMainWin, vCUDA**

Advantage: It has very low implementation effort

Shortcoming & limitation: poor application flexibility and isolation

Virtualization with Middleware/Library Support

Table 3.4 Middleware and Library Support for Virtualization

Middleware or Runtime Library and References or Web Link	Brief Introduction and Application Platforms
WABI (http://docs.sun.com/app/docs/doc/802-6306)	Middleware that converts Windows system calls running on x86 PCs to Solaris system calls running on SPARC workstations
Lxrun (Linux Run) (http://www.ugcs.caltech.edu/~steven/lxrun/)	A system call emulator that enables Linux applications written for x86 hosts to run on UNIX systems such as the SCO OpenServer
WINE (http://www.winehq.org/)	A library support system for virtualizing x86 processors to run Windows applications under Linux, FreeBSD, and Solaris
Visual MainWin (http://www.mainsoft.com/)	A compiler support system to develop Windows applications using Visual Studio to run on Solaris, Linux, and AIX hosts
vCUDA (Example 3.2) (IEEE <i>IPDPS</i> 2009 [57])	Virtualization support for using general-purpose GPUs to run data-intensive applications under a special guest OS

User-Application level :

- It virtualizes an application as a virtual machine.
- This layer sits as an application program on top of an operating system and exports an abstraction of a VM that can run programs written and compiled to a particular abstract machine definition.
- Also known as process-level virtualization
- Typical systems:
JVM , NET CLR , Panot

Advantage: has the best application isolation

Shortcoming & limitation: low performance, low application flexibility and high implementation complexity.

Table 3.1 Relative Merits of Virtualization at Various Levels

Level of Implementation	Higher Performance	Application Flexibility	Implementation Complexity	Application Isolation
ISA	X	XXXXX	XXX	XXX
Hardware-level virtualization	XXXXX	XXX	XXXXX	XXXX
OS-level virtualization	XXXXX	XX	XXX	XX
Runtime library support	XXX	XX	XX	XX
User application level	XX	XX	XXXXX	XXXXX

Full Virtualization vs. Para-Virtualization

Full virtualization

- Does not need to modify guest OS, and critical instructions are emulated by software through the use of binary translation.
- On the other hand, para virtualization needs to modify guest OS, and non-virtualizable instructions are replaced by hypercalls that communicate directly with the hypervisor or VMM.
- As of full virtualization, the advantage is no need to modify OS. However, this approach of binary translation slows down the performance a lot.

Para virtualization

- Reduces the overhead, but the cost of maintaining paravirtualized OS is high. The improvement depends on the workload. VMware Workstation applies full virtualization, which uses binary translation to automatically modify x86 software on-the-fly to replace critical instructions.
- The para virtualization is supported by Xen, Denali and VMware ESX.

Full Virtualization

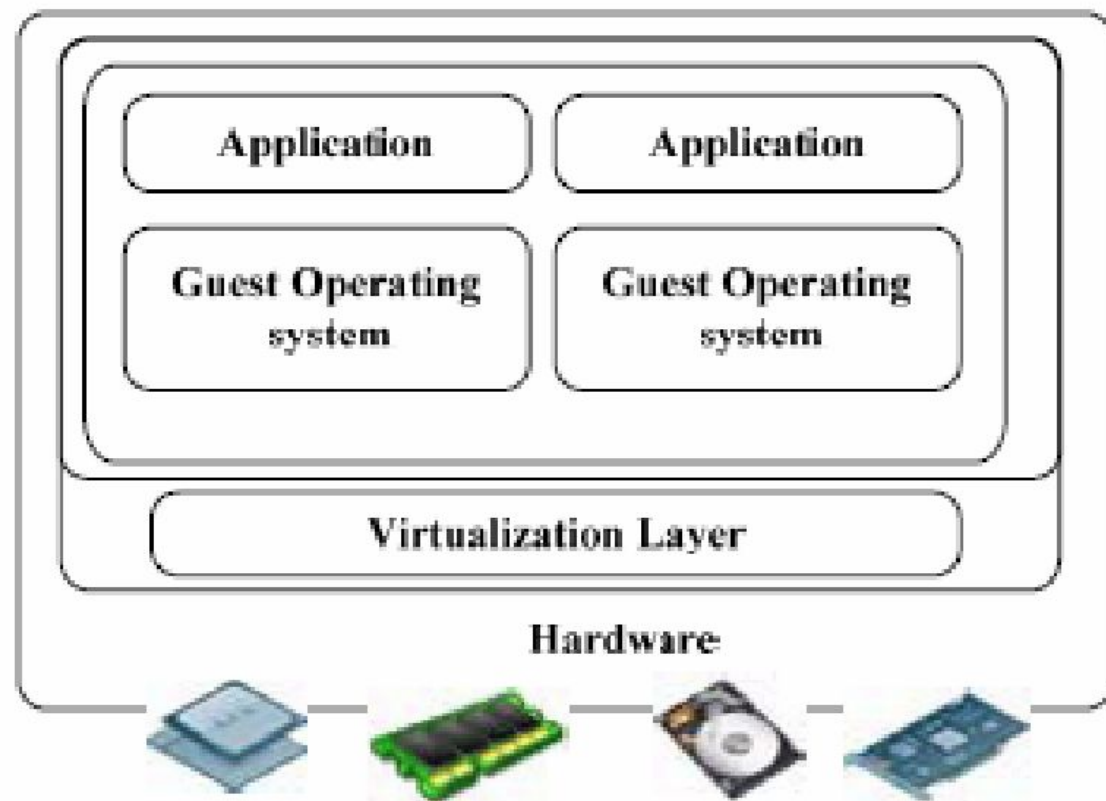


Figure 6.9 The concept of full virtualization using a hypervisor or a VMM directly sitting on top of the bare hardware devices. Note that no host OS is used here as in Figure 6.11.

Major VMM and Hypervisor Providers

VMM Provider	Host CPU	Guest CPU	Host OS	Guest OS	VM Architecture
VMware Work-station	X86, x86-64	X86, x86-64	Windows, Linux	Windows, Linux, Solaris, FreeBSD, Netware, OS/2, SCO, BeOS, Darwin	Full Virtualization
VMware ESX Server	X86, x86-64	X86, x86-64	No host OS	The same as VMware workstation	Para-Virtualization
XEN	X86, x86-64, IA-64	X86, x86-64, IA-64	NetBSD, Linux, Solaris	FreeBSD, NetBSD, Linux, Solaris, windows XP and 2003 Server	Hypervisor
KVM	X86, x86-64, IA64, S390, PowerPC	X86, x86-64, IA64, S390, PowerPC	Linux	Linux, Windows, FreeBSD, Solaris	Para-Virtualization

Hypervisor

A hypervisor is a hardware virtualization technique allowing multiple operating systems, called guests to run on a host machine. This is also called the Virtual Machine Monitor (VMM).

Type 1 hypervisor or the bare metal hypervisor sits on the bare metal computer hardware like the CPU, memory, etc. All the guest operating systems are a layer above the hypervisor. So the hypervisor is the first layer over the hardware. The original CP/CMS hypervisor developed by IBM was of this kind. Examples are Microsoft Hyper-V.

Type 2 or the hosted hypervisor do not run over the bare metal hardware but they run over a host operating system. The hypervisor is the second layer over the hardware. The guest operating systems run a layer over the hypervisor and so they form the third layer. Examples are FreeBSD. The operating system is usually unaware of the virtualization

Virtualization Structures/ Tools and Mechanisms:

- Before virtualization, the OS manages the hardware.
- After virtualization, A virtualization layer is inserted between the hardware and the OS.
- In such a case, the virtualization layer is responsible for converting portions of the real hardware into virtual hardware.
- Therefore different OSes can run on the same physical m/c simultaneously.
- Depending on the position of the virtualization layer, there are several classes of VM architectures, hypervisor, para-V, host-based V, VMM etc....

Hypervisor and the XEN Architecture

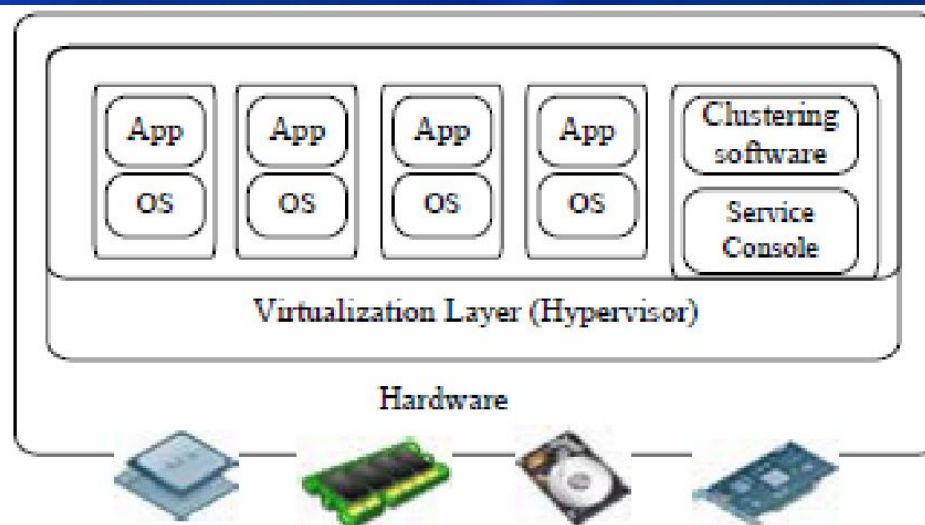


Figure 6.7 A hypervisor is the software layer for virtualization of the bare metal hardware. This layer can be implemented as a micro-kernel of the OS. It converts physical devices into virtual resources for user applications to run on the virtual machines deployed.

The hypervisor supports hardware-level virtualization (see Figure 3.1(b)) on bare metal devices like CPU, memory, disk and network interfaces. The hypervisor software sits directly between the physical hardware and its OS. This virtualization layer is referred to as either the VMM or the hypervisor. The hypervisor provides *hypercalls* for the guest OSes and applications. Depending on the functionality, a hypervisor can assume a *micro-kernel architecture* like the Microsoft Hyper-V. Or it can assume a *monolithic hypervisor architecture* like the VMware ESX for server virtualization.

The XEN Architecture (1)

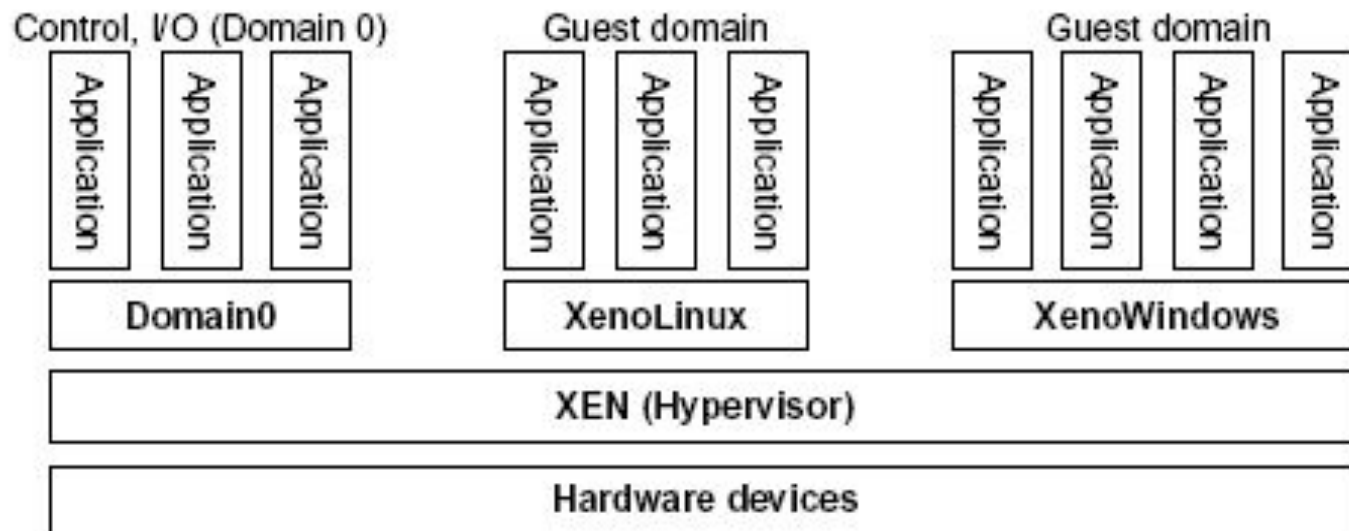


FIGURE 3.5

The Xen architecture's special domain 0 for control and I/O, and several guest domains for user applications.

The XEN Architecture (2)

Xen is an open source hypervisor program developed by Cambridge University. Xen is a micro-kernel hypervisor, which separates the policy from the mechanism. The Xen hypervisor implements all the mechanisms, leaving the policy to be handled by Domain 0, as shown in Figure 3.5. Xen does not include any device drivers natively [7]. It just provides a mechanism by which a guest OS can have direct access to the physical devices. As a result, the size of the Xen hypervisor is kept rather small. Xen provides a virtual environment located between the hardware and the OS. A number of vendors are in the process of developing commercial Xen hypervisors, among them are Citrix XenServer [62] and Oracle VM [42].

The XEN Architecture (3)

The core components of a Xen system are the hypervisor, kernel, and applications. The organization of the three components is important. Like other virtualization systems, many guest OSes can run on top of the hypervisor. However, not all guest OSes are created equal, and one in particular controls the others. The guest OS, which has control ability, is called Domain 0, and the others are called Domain U. Domain 0 is a privileged guest OS of Xen. It is first loaded when Xen boots without any file system drivers being available. Domain 0 is designed to access hardware directly and manage devices. Therefore, one of the responsibilities of Domain 0 is to allocate and map hardware resources for the guest domains (the Domain U domains).

For example, Xen is based on Linux and its security level is C2. Its management VM is named Domain 0, which has the privilege to manage other VMs implemented on the same host. If Domain 0 is compromised, the hacker can control the entire system. So, in the VM system, security policies are needed to improve the security of Domain 0. Domain 0, behaving as a VMM, allows users to create, copy, save, read, modify, share, migrate, and roll back VMs as easily as manipulating a file, which flexibly provides tremendous benefits for users. Unfortunately, it also brings a series of security problems during the software life cycle and data lifetime.

Para-Virtualization

In para-virtualization, the guest operating system has to be modified. Para-virtualization provides specially defined 'hooks' to do some tasks in the host and guest operating systems, which would otherwise have been done in a virtual environment, which is slower. The VMM in a para-virtualized platform is simpler because the critical tasks are now performed in the operating system rather than by the VMM. Since the virtualization overhead decreases the performance increases.

Some of the disadvantages are the compatibility and the portability is reduced because of the modified operating system. Also the cost of maintenance is high because of the deep OS modifications.

Some examples of Para-virtualization are **KVM and the XenWindowsGplPv project.**

Para-Virtualization with Compiler Support.

The KVM builds offers kernel-based VM on the Linux platform, based on para-virtualization

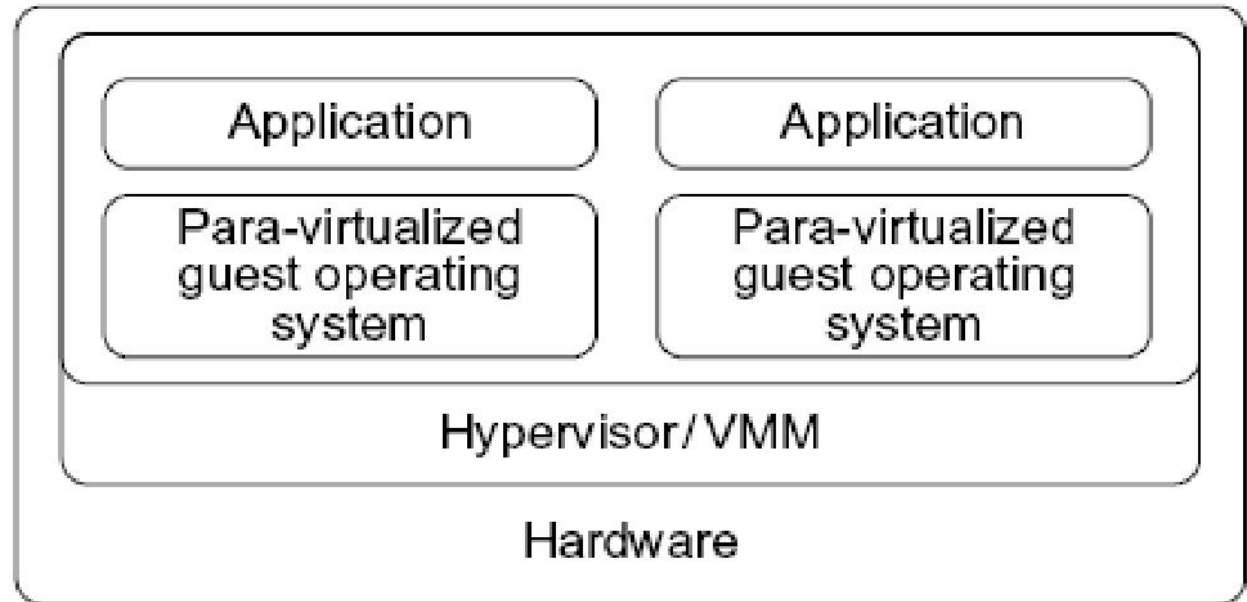


FIGURE 3.7

Para-virtualized VM architecture, which involves modifying the guest OS kernel to replace nonvirtualizable instructions with hypercalls for the hypervisor or the VMM to carry out the virtualization process (See Figure 3.8 for more details).

Binary Translation of Guest OS Requests using a VMM:

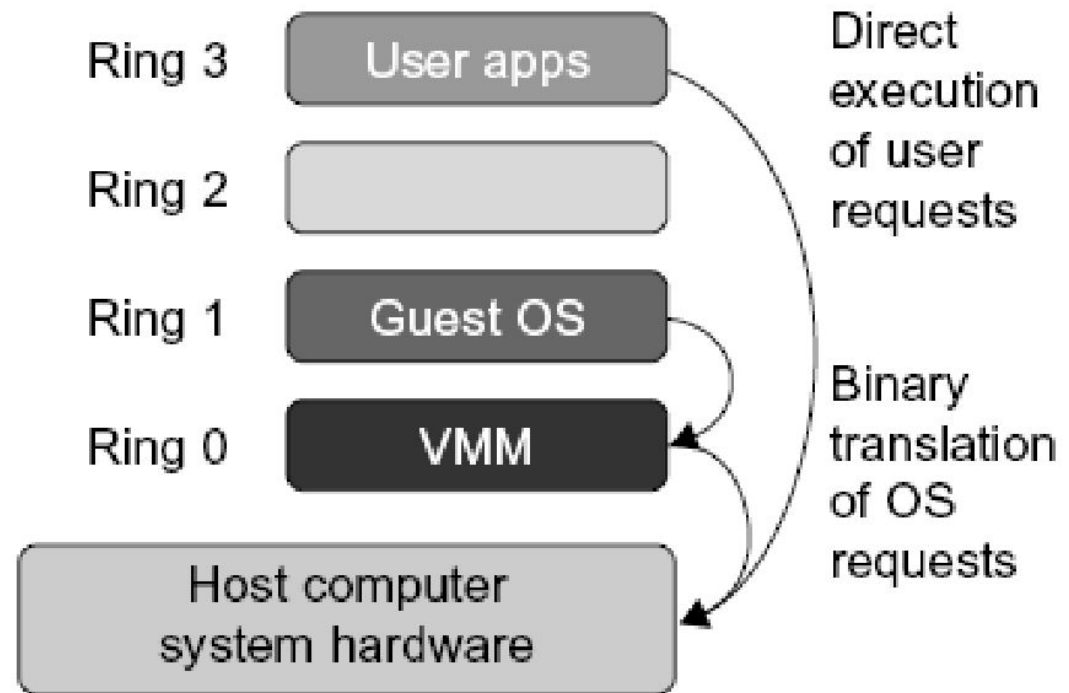


FIGURE 3.6

Indirect execution of complex instructions via binary translation of guest OS requests using the VMM plus direct execution of simple instructions on the same host.

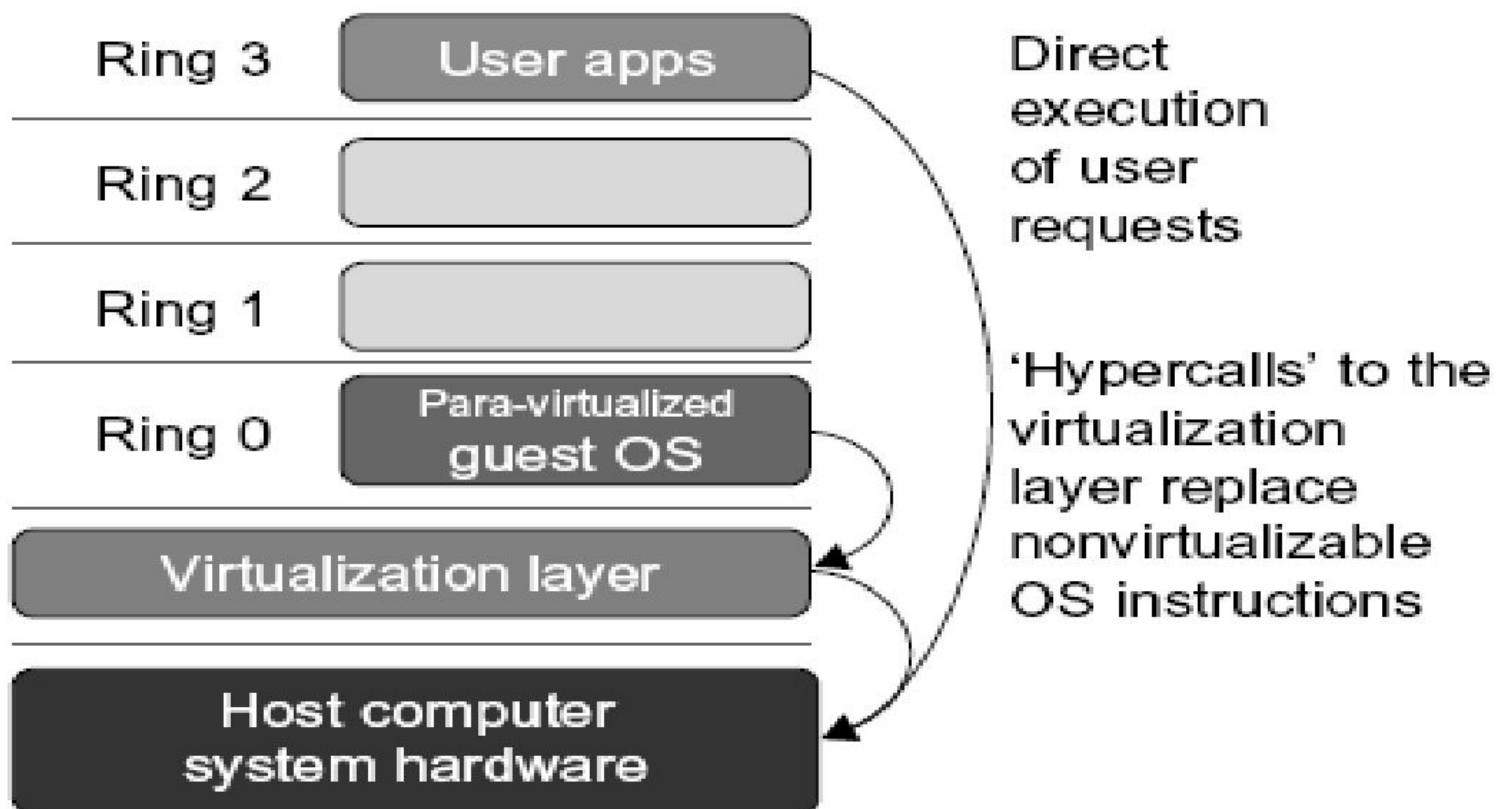


FIGURE 3.8

The Use of a para-virtualized guest OS assisted by an intelligent compiler to replace nonvirtualizable OS instructions by hypercalls.

Hypercall Execution

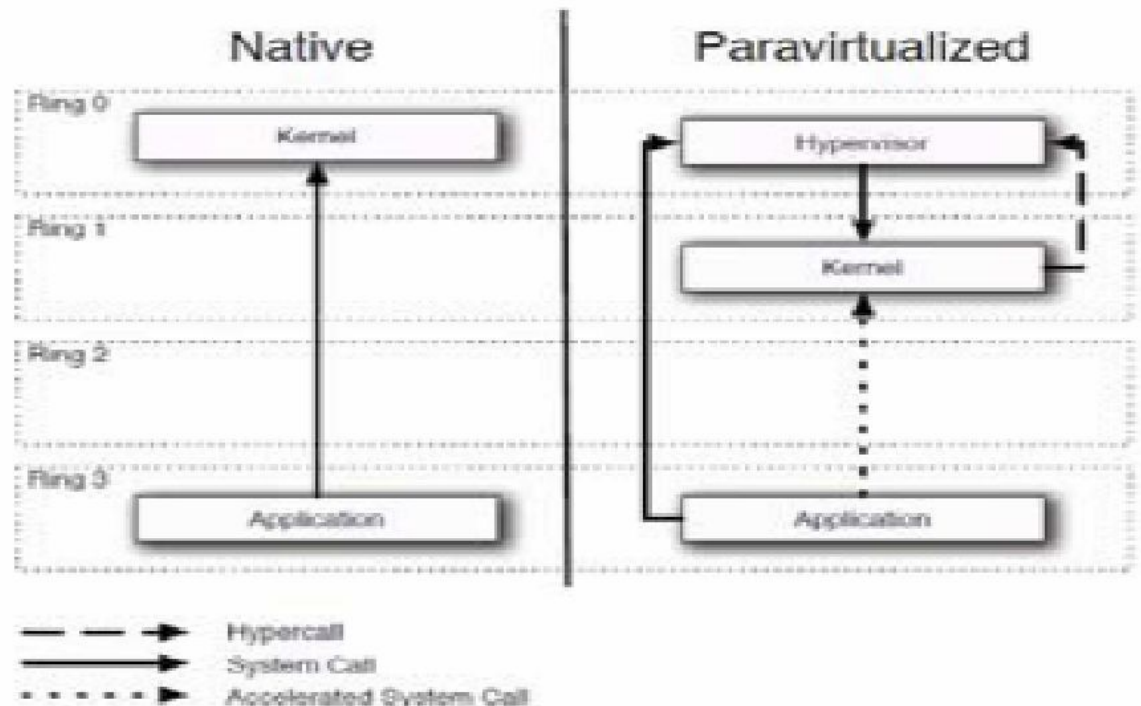


Figure 6.15. When an application VM issues a guest OS system call, it interrupts the hypervisor to handle and then pass control back to the guest OS. (Courtesy of “the definitive guide to the XEN hypervisor” by David Chisnall [13]).

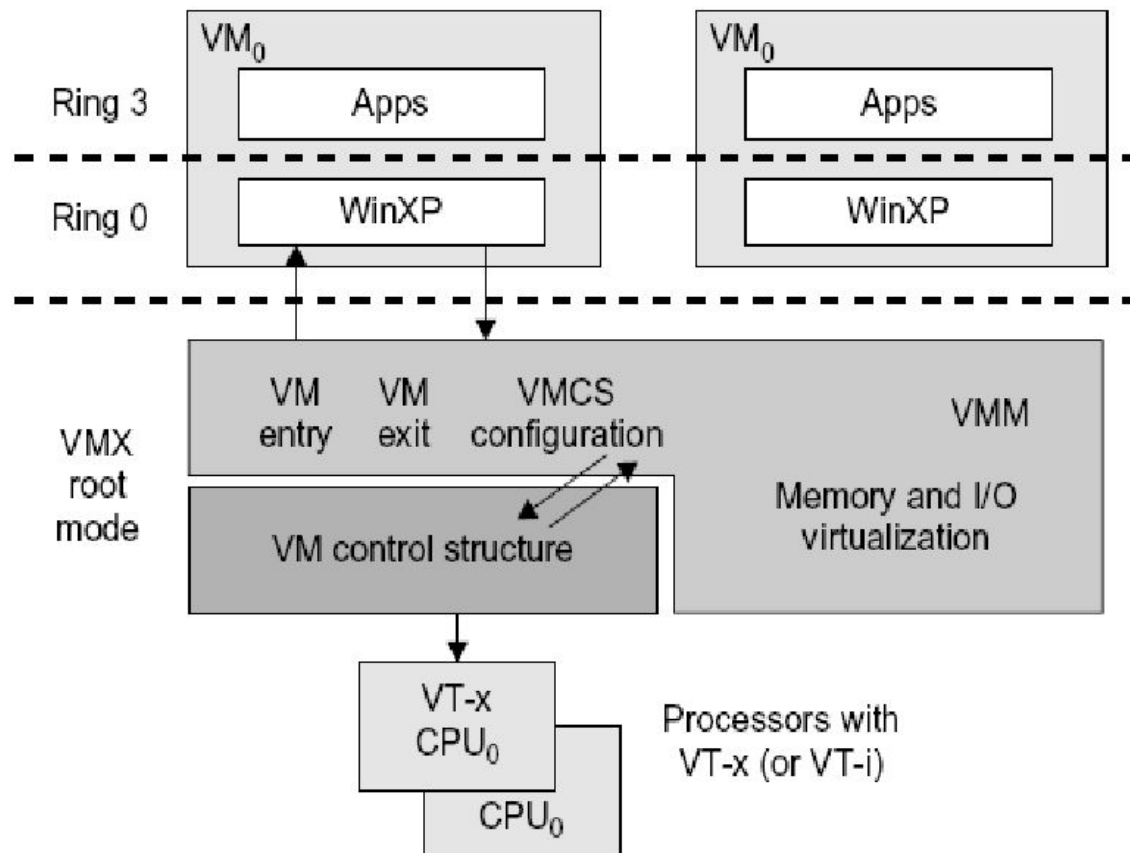


FIGURE 3.11

Intel Hardware-assisted CPU virtualization.

(Modified from [68], Courtesy of Lizhong Chen, USC)

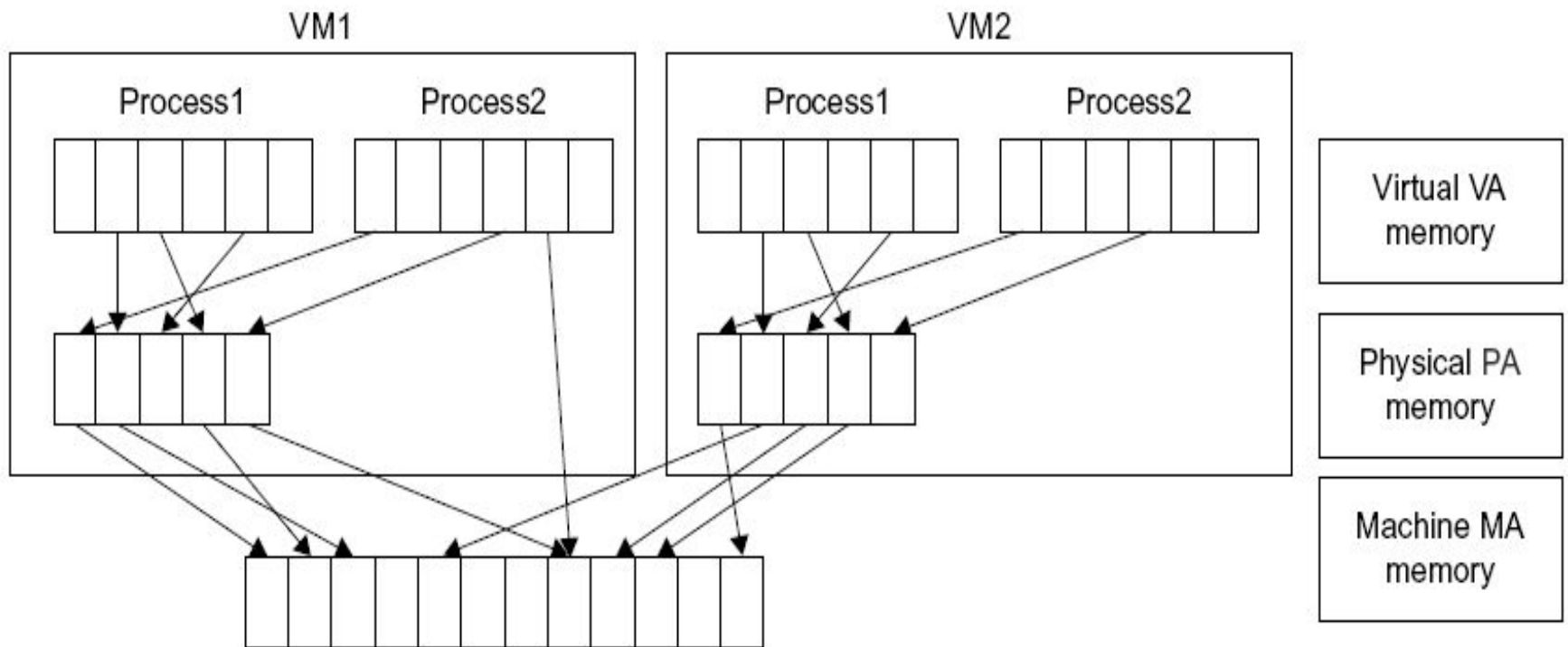
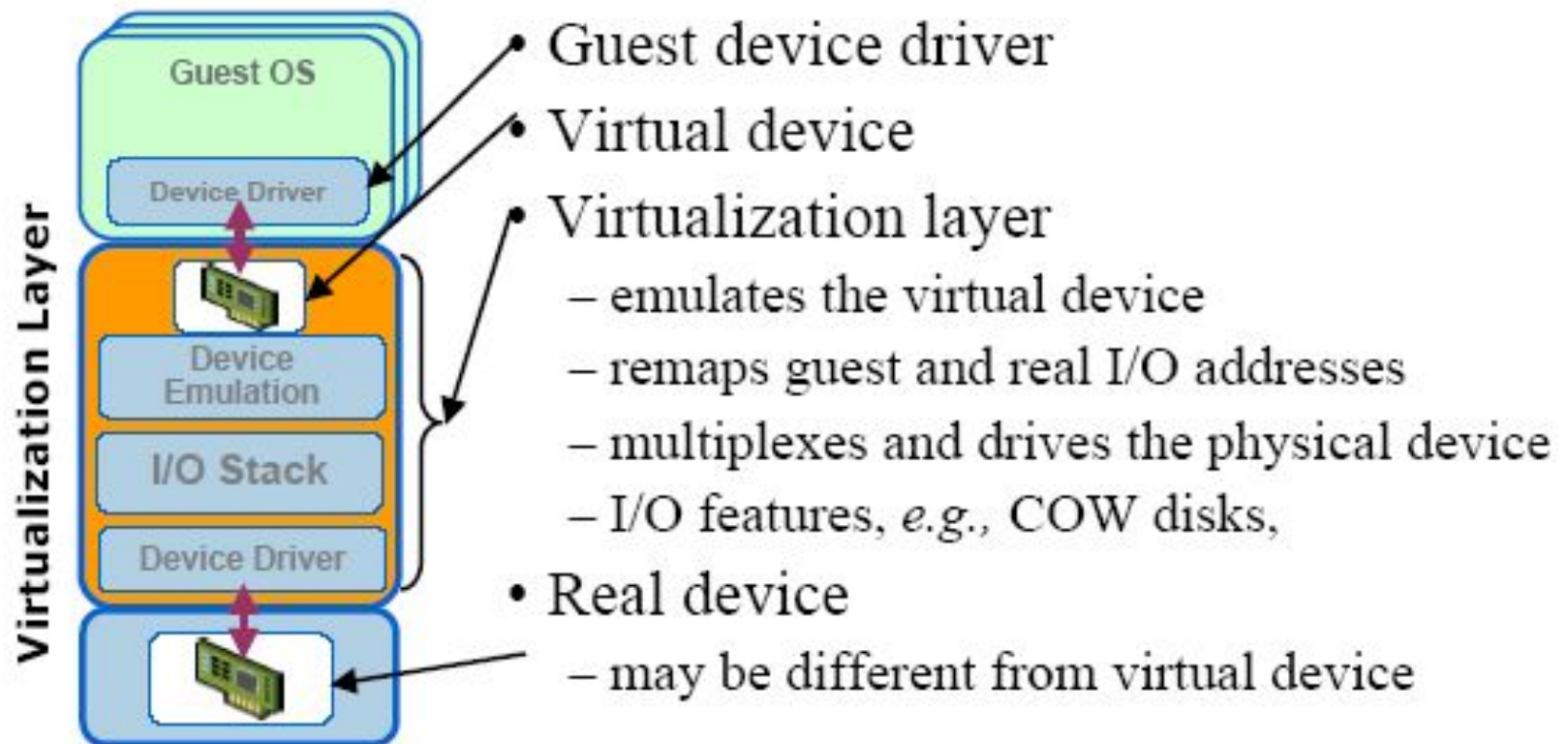


FIGURE 3.12

Two-level memory mapping procedure.

Current virtual I/O devices



CPU, Memory and I/O Virtualization :

- CPU virtualization demands hardware-assisted traps of sensitive instructions by the VMM
- Memory virtualization demands special hardware support (shadow page tables by VMWare or extended page table by Intel) to help translate virtual address into physical address and machine memory in two stages.
- I/O virtualization is the most difficult one to realize due to the complexity of I/O service routines and the emulation needed between the guest OS and host OS.

Multi-Core Virtualization:

VCPU vs. traditional CPU

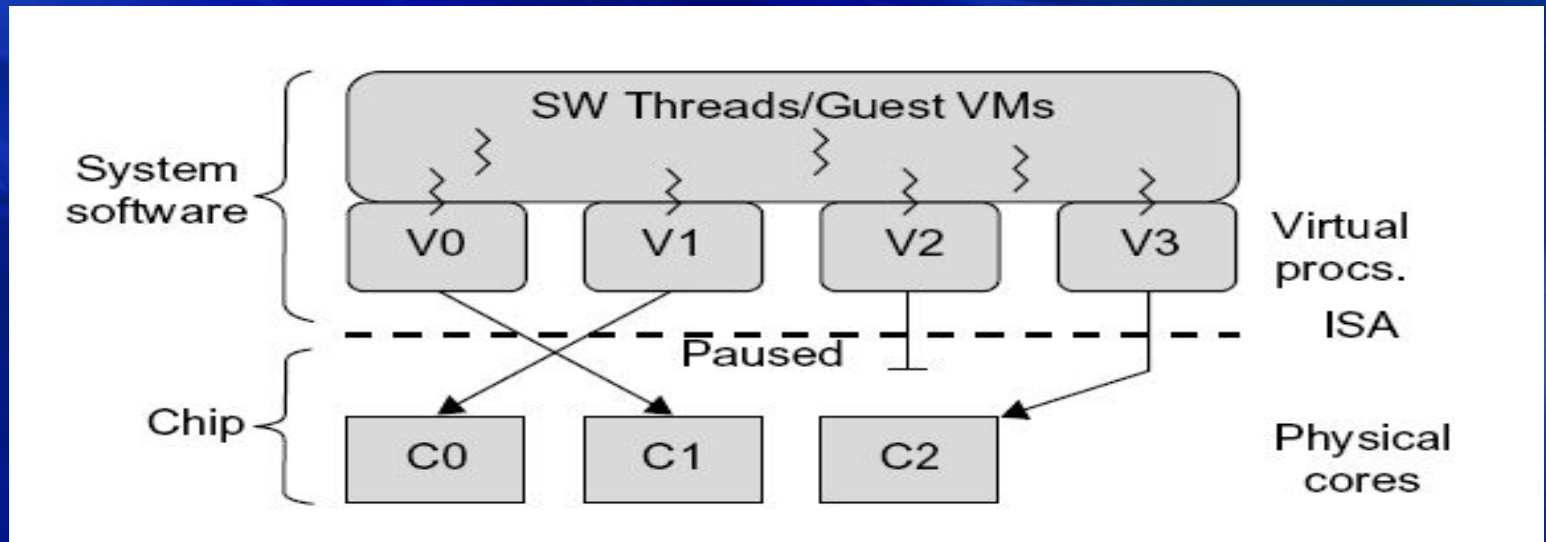


Figure 3.16 Four VCPUs are exposed to the software, whereas, only three cores are actually present. VCPUs V0, V1, and V3 have been transparently migrated, while VCPU V2 has been transparently suspended. (Courtesy of Wells, et al., “Dynamic Heterogeneity and the Need for Multicore Virtualization”, *ACM SIGOPS Operating Systems Review*, ACM Press, 2009 [68])

Virtual Cluster Characteristics

- The virtual cluster nodes can be either physical or virtual machines. Multiple VMs running with different OSs can be deployed on the same physical node.
- A VM runs with a guest OS, which is often different from the host OS, that manages the resources in the physical machine, where the VM is implemented.
- The purpose of using VMs is to consolidate multiple functionalities on the same server. This will greatly enhance the server utilization and application flexibility.
- VMs can be colonized (replicated) in multiple servers for the purpose of promoting distributed parallelism, fault tolerance, and disaster recovery.
- The size (number of nodes) of a virtual cluster can grow or shrink dynamically, similarly to the way an overlay network varies in size in a P2P network.
- The failure of any physical nodes may disable some VMs installed on the failing nodes. But the failure of VMs will not pull down the host system.

Virtual Cores vs. Physical Processor Cores

Physical cores	Virtual cores
The actual physical cores present in the processor.	There can be more virtual cores visible to a single OS than there are physical cores.
More burden on the software to write applications which can run directly on the cores.	Design of software becomes easier as the hardware assists the software in dynamic resource utilization.
Hardware provides no assistance to the software and is hence simpler.	Hardware provides assistance to the software and is hence more complex.
Poor resource management.	Better resource management.
The lowest level of system software has to be modified.	The lowest level of system software need not be modified.

Virtual Clusters vs. Physical Clusters

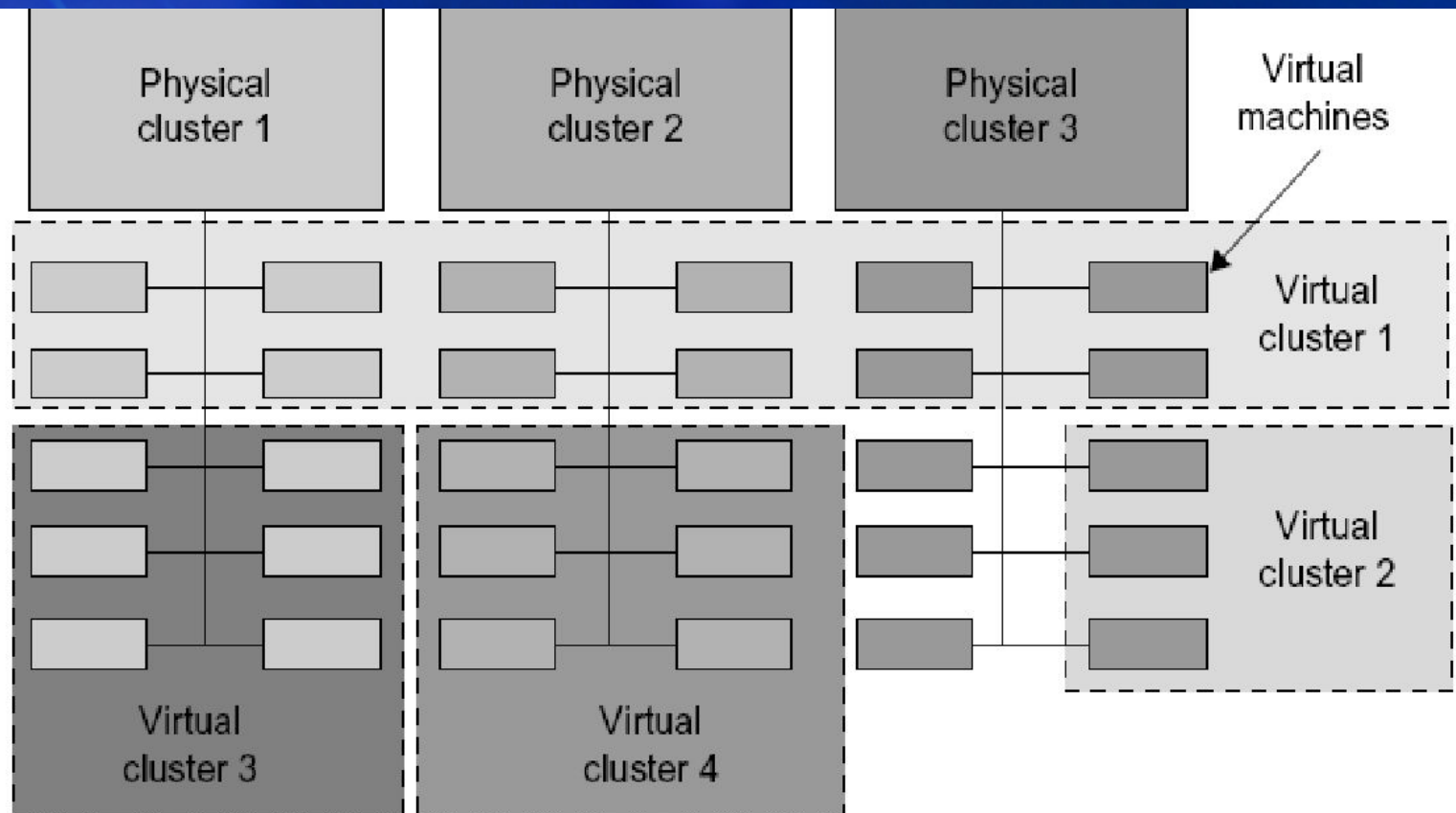


FIGURE 3.18

A cloud platform with 4 virtual clusters over 3 physical clusters shaded differently.

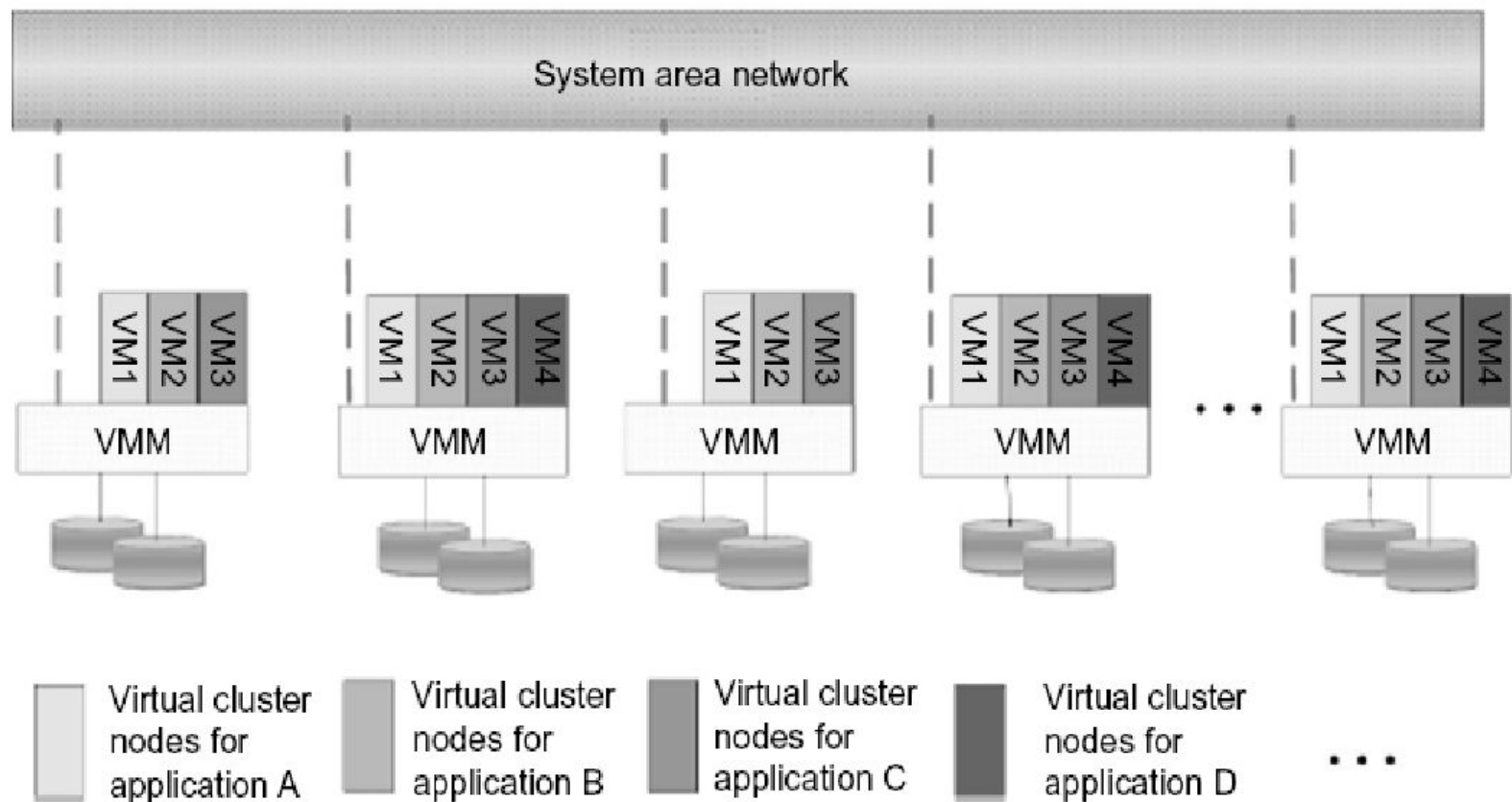


FIGURE 3.19

The concept of a virtual cluster based on application partitioning.

(Courtesy of Kang, Chen, Tsinghua University 2008)

Live Migration of Virtual Machines

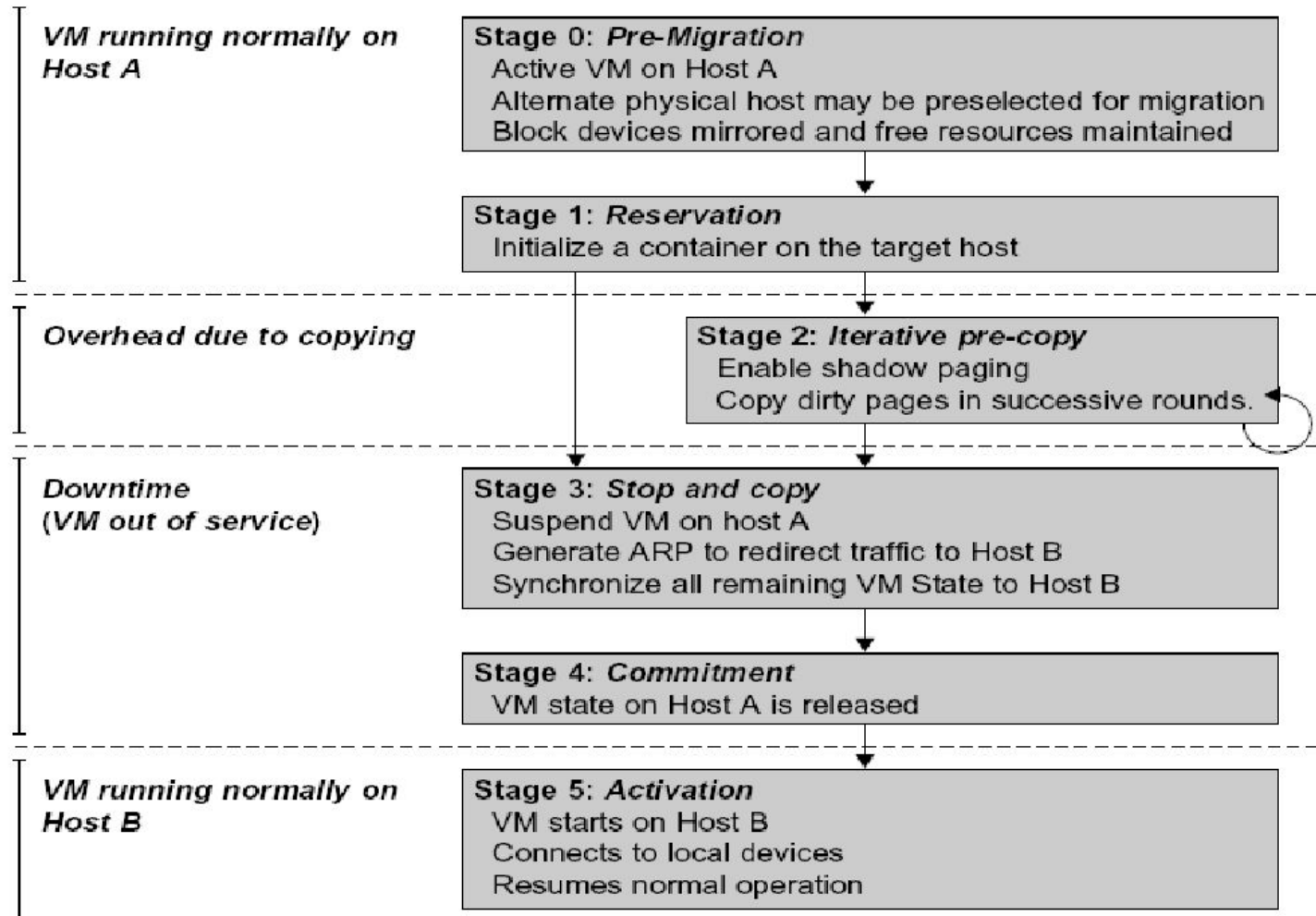


FIGURE 3.20

Live migration process of a VM from one host to another.

(Courtesy of C. Clark, et al. [14])

Cloud OS for Building Private Clouds

Table 3.6 VI Managers and Operating Systems for Virtualizing Data Centers [9]

Manager/ OS, Platforms, License	Resources Being Virtualized, Web Link	Client API, Language	Hypervisors Used	Public Cloud Interface	Special Features
Nimbus Linux, Apache v2	VM creation, virtual cluster, www .nimbusproject.org/	EC2 WS, WSRF, CLI	Xen, KVM	EC2	Virtual networks
Eucalyptus Linux, BSD	Virtual networking (Example 3.12 and [41]), www .eucalyptus.com/	EC2 WS, CLI	Xen, KVM	EC2	Virtual networks
OpenNebula Linux, Apache v2	Management of VM, host, virtual network, and scheduling tools, www.opennebula.org/	XML-RPC, CLI, Java	Xen, KVM	EC2, Elastic Host	Virtual networks, dynamic provisioning
vSphere 4 Linux, Windows, proprietary	Virtualizing OS for data centers (Example 3.13), www .vmware.com/ products/vsphere/ [66]	CLI, GUI, Portal, WS	VMware ESX, ESXi	VMware vCloud partners	Data protection, vStorage, VMFS, DRM, HA

Eucalyptus : An Open-Source OS for Setting Up and Managing Private Clouds

Eucalyptus is an open source software system (Figure 3.27) intended mainly for supporting Infrastructure as a Service (IaaS) clouds. The system primarily supports virtual networking and the management of VMs; virtual storage is not supported. Its purpose is to build private clouds that can interact with end users through Ethernet or the Internet. The system also supports interaction with other private clouds or public clouds over the Internet. The system is short on security and other desired features for general-purpose grid or cloud applications.

- **Instance Manager** controls the execution, inspection, and terminating of VM instances on the host where it runs.
- **Group Manager** gathers information about and schedules VM execution on specific instance managers, as well as manages virtual instance network.
- **Cloud Manager** is the entry-point into the cloud for users and administrators. It queries node managers for information about resources, makes scheduling decisions, and implements them by making requests to group managers.